Datamorpho Specification v0.001

Normative Core Specification

Author: German Abal <g@evvm.org>
Date: 03/20/2026
Status: Draft
Intended category: Open technical specification, suitable for later public standardization discussion
Language of this specification: English

---

1. Introduction

Datamorpho is a file format and protocol extension model that allows a valid ordinary file to contain one or more concealed alternate states while preserving a visible base representation.

A Datamorphed file:

- remains a valid carrier file,
- publicly declares that Datamorphosis exists,
- publicly declares expected triggers and MorphoStorage references,
- conceals reconstruction instructions,
- and may contain one or more hidden states embedded as encrypted, fragmented, sparsely arranged, and optionally chaff-interleaved payload material.

This specification defines the **normative core** of Datamorpho version `0.001`.

This version standardizes:

- the core object model,
- the public manifest,
- the reconstruction object,
- the digest-binding model,
- the JPEG carrier profile,
- the TXT carrier profile,
- the PDF carrier profile,
- payload offset semantics,
- versioning rules,

- and minimum conformance requirements.

This version does **not** standardize:

- external key distribution or access-control infrastructure.

However, in Datamorpho v0.001, a reconstruction object **MUST** contain sufficient secret material, directly or by fragment override, to reconstruct the target hidden state.

---

2. Conformance Language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as normative requirement terms.

---

3. Scope

Datamorpho v0.001 defines:

1. a **public declaration layer**,
2. a **concealed payload layer**,
3. a **private reconstruction layer**,
4. a **carrier profile model** for JPEG, TXT, and PDF,
5. a **digest cross-binding model** between public state descriptors and reconstruction objects,
6. and a **versioned, implementation-neutral structure** suitable for future interoperable creators and viewers.

Datamorpho v0.001 is creation-oriented. Viewer behavior is only defined where necessary to interpret the file structure.

---

4. Terminology

4.1 Base File

The ordinary visible file used as the initial carrier input before Datamorpho embedding.

4.2 Carrier File

The final Datamorphed file that conforms to one of the carrier profiles defined in this specification.

4.3 Datamorphed File

A carrier file that contains a Datamorpho public manifest and concealed payload material.

4.4 Hidden State

A concealed alternate content state associated with a Datamorphed file.

4.5 Fragment

A unit of concealed payload data that MAY be independently encrypted, placed, and reconstructed.

4.6 Trigger

A declared condition associated with a hidden state.

4.7 MorphoStorage

A locator, reference, or instruction set describing where to look for the reconstruction object of a hidden state.

4.8 Public Manifest

The public, intentionally visible metadata describing Datamorpho presence, version, states, triggers, MorphoStorage references, and reconstruction-object digests.

4.9 Reconstruction Object

A private or selectively released metadata object that contains the information needed to reconstruct exactly one hidden state.

4.10 Payload Region

A byte region that contains concealed payload material. A file MAY contain one or more payload regions, depending on carrier profile.

4.11 Chaff

Bytes intentionally present in a payload region but not referenced by a given reconstruction object for the purpose of increasing ambiguity and attack cost.

4.12 Layout Strategy

The arrangement pattern used for meaningful bytes within a payload region.

4.13 Carrier Profile

The file-family-specific rules for embedding Datamorpho content.

4.14 Viewer / Interpreter

Software that parses a Datamorphed file and, when authorized and able, reconstructs one or more hidden states.

4.15 Key Material

Secret material included in a reconstruction object and required to recover one or more fragments or a whole hidden state. The interpretation of that material is defined by the corresponding `crypto_suite`.

---

5. Design Principles

The following principles are normative for the design of Datamorpho v0.001:

1. The base file **MUST** remain a valid ordinary file for its carrier type.
2. Public metadata **MUST** be separated from reconstruction metadata.
3. Reconstruction metadata **MUST NOT** be required to be stored inside the public carrier.
4. Payload offsets in reconstruction objects **MUST** refer to canonical decoded payload bytes.
5. Cryptographic algorithms are **not fixed by this specification**.
6. Implementations **MAY** mix cryptographic suites across fragments or states.
7. Layout strategy in v0.001 **MUST** be either `"sparse"` or `"sparse-with-chaff"`.
8. Bytes not referenced by a given reconstruction object **MAY** be arbitrary, including chaff or bytes referenced by other hidden states.
9. Obfuscation in Datamorpho v0.001 is primarily a **payload-layout property**, not a requirement to transform encrypted fragment bytes.
10. A reconstruction object **MUST** include sufficient key material to reconstruct the target hidden state.
11. Different fragments within the same hidden state **MAY** use different cryptographic suites and different key material.
12. Different hidden states **MAY** use different cryptographic suite combinations.

---

6. Identifier Rules

Unless otherwise stated, the following identifiers:

- `file_id`
- `state_id`
- `trigger_id`
- `reconstruction_id`
- `fragment_id`
- `payload_region`

MUST match this pattern:

^[a-z0-9][a-z0-9._-]{0,63}$

Identifiers are case-sensitive. Writers **SHOULD** use lowercase ASCII only.

---

7. Common Serialization Rules

7.1 Character Encoding

All JSON defined by this specification **MUST** be encoded as UTF-8 without BOM.

7.2 JSON Rules

For all JSON objects defined in this specification:

- duplicate keys **MUST NOT** appear,
- parsers **MUST** accept any member order,
- writers **SHOULD** emit members in stable deterministic order,
- unknown keys **MUST** be ignored unless a future version states otherwise,
- extension keys **SHOULD** begin with `x-`.

7.3 Integers

This rule applies only to **binary integer fields** defined by this specification.

All such binary integer fields **MUST** be unsigned and **MUST** use big-endian byte order.

JSON numeric values are textual and are not affected by this rule.

7.4 Base64url

Any binary value carried in JSON **MUST** use base64url encoding without padding unless explicitly stated otherwise.

7.5 Newlines

For textual Datamorpho envelope sections, writers **MUST** emit `LF` line endings. Parsers **MUST** accept either `LF` or `CRLF`.

7.6 Digest Object

Wherever a digest is used in this specification, it **MUST** be represented as:

```
{
 "algorithm": "sha256",
 "value": "base64url-without-padding"
}
```

The `algorithm` value **MUST** be explicit. This specification does not restrict digest algorithms to a single family, but all conforming implementations **MUST** name the algorithm used.

7.7 Key Material Object

```
{
"format": "raw|suite-defined|custom",
"encoding": "base64url",
"value": "base64url-without-padding"
}
```

Required fields

- `format`
  MUST be one of:
    - `"raw"`
    - `"suite-defined"`
    - `"custom"`
- `encoding`
  - MUST be `"base64url"` in v0.001
- `value`
  - MUST be a base64url string without padding

Optional fields

- `description`
- `x-*`

The contents of `key_material` are interpreted according to the corresponding `crypto_suite`. In some implementations this may be a raw key; in others it may be a suite-defined secret blob or key package.

7.8 Canonical JSON Serialization for Digests

When this specification requires hashing of a JSON object, the hash input **MUST** be the UTF-8 bytes of the object serialized in this canonical form:

1. object keys sorted lexicographically by Unicode code point,
2. arrays preserved in their original order,
3. no insignificant whitespace,
4. strings encoded as standard JSON strings,
5. integers serialized in base-10 without leading `+`,
6. zero serialized as `0`.

7.9 Excluded Fields for Reconstruction Digest

The `reconstruction_digest` of a reconstruction object **MUST** be computed over the canonical JSON serialization of that reconstruction object **after removing** these fields if present:

- `carrier_file_digest`
- `base_file_digest`

This exclusion rule prevents circular dependency between file binding and reconstruction-object binding.

---

8. Logical Datamorpho Model

A Datamorphed file logically consists of:

1. a **base file**,
2. a **public manifest**,
3. one or more **payload regions**,

4. and zero or more **external reconstruction objects**.

A Datamorphed file conforming to this specification **MUST** contain exactly one public manifest and at least one non-empty payload region.

A Datamorpho deployment **MAY** use one reconstruction object per state, one reconstruction object per release event, or any equivalent split-publication arrangement, provided that each reconstruction object identifies exactly one target hidden state.

---

9. Public Manifest

The public manifest **MUST** be a JSON object.

9.1 Required Top-Level Fields

```
{
 "datamorpho_version": "0.001",
 "manifest_type": "public",
 "carrier": "jpeg|txt|pdf",
 "profile": "jpeg-trailer|txt-envelope|pdf-incremental",
 "states": []
}
```

Field Requirements

- `datamorpho_version`
  - **MUST** be the string `"0.001"`.
- `manifest_type`
  - **MUST** be the string `"public"`.
- `carrier`
  - **MUST** be one of:
    - `"jpeg"`
    - `"txt"`
    - `"pdf"`
- `profile`
  - **MUST** be one of:
    - `"jpeg-trailer"`
    - `"txt-envelope"`
    - `"pdf-incremental"`
- `states`
  - **MUST** be a non-empty array of state descriptor objects.

9.2 Optional Top-Level Fields

The following top-level fields are OPTIONAL:

- `file_id`
- `title`
- `description`
- `notes`
- `x-*`

If `file_id` is present, it **MUST** conform to Section 6.

9.3 State Descriptor Object

Each member of `states` **MUST** be an object with the following required fields:

```
{
 "state_id": "state-1",
 "triggers": [],
 "morphostorage": [],
 "reconstruction_digest": {
   "algorithm": "sha256",
   "value": "base64url..."
 }
}
```

Required Fields

- `state_id`
  - **MUST** conform to Section 6.
- `triggers`
  - **MUST** be a non-empty array of trigger descriptor objects.
- `morphostorage`
  - **MUST** be a non-empty array of MorphoStorage descriptor objects.
- `reconstruction_digest`
  - **MUST** be a digest object as defined in Section 7.6.
  - **MUST** be the digest of the corresponding reconstruction object computed according to Section 7.8.

Optional Fields

- `label`
- `description`

- `x-*`

9.4 MorphoStorage Descriptor Object

Each MorphoStorage descriptor **MUST** be an object with:

```
{
 "type": "uri|text|evm|custom",
 "value": "implementation-defined locator or instruction"
}
```

Required Fields

- `type`
  - **MUST** be one of:
    - `"uri"`
    - `"text"`
    - `"evm"`
    - `"custom"`
- `value`
  - **MUST** be a string.

Optional Fields

- `chain_id`
  - **MAY** be a non-negative integer.
- `contract_address`
  - **MAY** be a string.
- `function`
  - **MAY** be a string.
- `description`
  - **MAY** be a string.
- `x-*`

9.5 Trigger Descriptor Object

Each trigger descriptor **MUST** be an object with:

```
{
 "trigger_id": "t1",
 "type": "time|event|action|ownership|custom",
 "value": "implementation-defined string"
}
```

Required Fields

- `trigger_id`
  - **MUST** conform to Section 6.
- `type`
  - **MUST** be one of:
    - `"time"`
    - `"event"`
    - `"action"`
    - `"ownership"`
    - `"custom"`
- `value`
  - **MUST** be a string.

Optional Fields

- `description`
- `x-*`

9.6 Trigger Semantics

Triggers in the public manifest are **descriptive declarations** in version `0.001`.

This specification does **not** define trigger execution semantics.

A parser **MUST NOT** assume that a trigger can be locally verified unless implementation-specific logic exists.

---

10. Reconstruction Object

A reconstruction object **MUST** be a JSON object.

A reconstruction object **MUST** identify exactly one target hidden state.

10.1 Top-Level Fields

```
{
 "datamorpho_version": "0.001",
 "object_type": "reconstruction",
 "reconstruction_id": "r1",
 "state_id": "state-1",
 "layout_strategy": "sparse|sparse-with-chaff",
 "carrier_file_digest": {
   "algorithm": "sha256",
   "value": "base64url..."
 },
```

```
 "fragments": [],
 "reassembly": "concat-by-order",
 "state_key_material": {
 "format": "raw|suite-defined|custom",
 "encoding": "base64url",
 "value": "..."
 }
}
```

Field Requirements

- `datamorpho_version`
  - **MUST** be `"0.001"`.
- `object_type`
  - **MUST** be `"reconstruction"`.
- `reconstruction_id`
  - **MUST** conform to Section 6.
- `state_id`
  - **MUST** conform to Section 6.
  - **MUST** identify a state declared in the public manifest.
- `layout_strategy`
  - **MUST** be one of:
    - `"sparse"`
    - `"sparse-with-chaff"`
- `carrier_file_digest`
  - **MUST** be a digest object as defined in Section 7.6.
  - **MUST** be the digest of the final Datamorphed carrier file bytes.
- `fragments`
  - **MUST** be a non-empty array of fragment descriptor objects.
- `reassembly`
  - **MUST** be a string.
  - v0.001 defines the value `"concat-by-order"`.
- `state_key_material`
  - **MAY** be present.
  - **MUST**, when present, be a **Key Material Object** as defined in Section 7.7.
  - **Acts as the default key material** for all fragments of the state.
  - **Is overridden by fragment-level `key_material` when present.**
  - Its `format`, `encoding`, and `value` fields MUST follow Section 7.7.

10.2 Optional Top-Level Fields

Optional fields include:

- `base_file_digest`

- `state_media_type`
- `state_filename`
- `integrity`
- `notes`
- `x-*`

If present:

- `base_file_digest`
    - **MUST** be a digest object as defined in Section 7.6.
    - **MAY** be used to identify the original base file prior to Datamorpho embedding.

10.3 Key Material Requirements

A reconstruction object in Datamorpho v0.001 **MUST** provide sufficient key material to reconstruct every fragment of the target hidden state.

This requirement **MAY** be satisfied in either of the following ways:

1. by providing `state_key_material` at the reconstruction-object level, which acts as the default key material for all fragments of that state;
2. by providing `key_material` at the fragment level;
3. or by using both, where fragment-level `key_material` overrides `state_key_material`.

If a fragment requires key material different from the shared state-level key material, that fragment **MUST** provide its own `key_material`.

10.4 Fragment Descriptor Object

Each fragment descriptor **MUST** contain:

```
{
 "fragment_id": "f1",
 "payload_region": "main",
 "offset": 59,
 "length": 442,
 "order": 1,
 "crypto_suite": "implementation-defined"
 }
}
```

Fields

- `fragment_id`
    - **MUST** conform to Section 6.

- `payload_region`
  - **MUST** conform to Section 6.
  - **MUST** identify the payload region from which bytes are read.
- `offset`
  - **MUST** be a non-negative integer.
  - **MUST** be interpreted as a zero-based byte offset into the canonical decoded payload region.
- `length`
  - **MUST** be a positive integer.
- `order`
  - **MUST** be a positive integer.
  - Fragment reconstruction order is ascending numeric order.
- `crypto_suite`
  - **MUST** be a string naming the cryptographic suite or transformation required to recover usable bytes for that fragment.
  - Fragment-level cryptographic suites **MAY differ within the same hidden state** and **MAY also differ across different hidden states**.
- `key_material`
  - **MAY** be present at fragment level. When present, it overrides `state_key_material` for that fragment.

Optional Fields

- `fragment_transform`
- `integrity`
- `x-*`

If `fragment_transform` is absent, implementations **MUST** treat it as `"none"`.

10.5 Fragment Order and Physical Placement

Fragments in a reconstruction object:

- **MAY** appear in any order in the JSON array,
- **MAY** reference payload spans in any physical byte order,
- **MUST** be logically reassembled by ascending `order`,
- and **MAY** be interleaved with chaff or with bytes referenced by other hidden states.

Example: a valid state may declare fragments located at offsets `59`, `20`, `900`, and `1`, with logical orders `1`, `2`, `3`, and `4`. Physical offset order is irrelevant. Logical reconstruction order is authoritative.

Fragments belonging to the same hidden state are **not required** to share a single cryptographic suite. A valid state **MAY** be reconstructed from fragments protected by heterogeneous cryptographic suites, provided the reconstruction object supplies sufficient per-fragment instructions and key material.

10.6 Fragment Overlap

Fragments within the same reconstruction object **SHOULD NOT** overlap within the same payload region.

If they do overlap, behavior is implementation-defined and such use is **NOT RECOMMENDED** in v0.001.

10.7 Integrity Object

If present, `integrity` **MUST** be a digest object as defined in Section 7.6.

10.8 Reconstruction Object Scope

A reconstruction object **MAY** be:

- private,
- selectively released,
- published on MorphoStorage,
- published later than the Datamorphed file,
- or split among systems, provided the final effective reconstruction data remains logically equivalent.

---

11. Canonical Payload Byte Semantics

This section is critical.

All offsets and lengths in reconstruction objects **MUST** refer to the **canonical decoded payload byte region**, not to transport encoding length.

This means:

- for **JPEG**, offsets refer to bytes in the appended Datamorpho binary payload region,
- for **TXT**, offsets refer to bytes obtained **after** decoding the textual payload encoding,
- for **PDF**, offsets refer to bytes inside the referenced payload stream object.

This rule is mandatory.

---

12. Layout Strategy Semantics

12.1 `sparse`

A reconstruction object with `layout_strategy: "sparse"` indicates that meaningful fragment bytes occupy non-contiguous spans of one or more payload regions and that bytes not referenced by that reconstruction object are irrelevant for reconstructing that state.

12.2 `sparse-with-chaff`

A reconstruction object with `layout_strategy: "sparse-with-chaff"` indicates that meaningful fragment bytes occupy non-contiguous spans of one or more payload regions and that bytes not referenced by that reconstruction object are intentionally present as ambiguity-increasing material.

Those unreferenced bytes **MAY** include:

- chaff,
- arbitrary filler,
- bytes belonging to other hidden states,
- or any combination thereof.

12.3 Layout Strategy and Obfuscation

In Datamorpho v0.001, layout strategy is the primary normative expression of obfuscation.

This specification does **not** require ciphertext-specific obfuscation transforms to fragment bytes. Implementations **MAY** use such transforms, but they are not the primary obfuscation model standardized here.

---

13. Datamorpho Binary Block (DMB-0.001)

The Datamorpho Binary Block is the required binary container for the JPEG profile.

13.1 Structure

A DMB-0.001 block **MUST** consist of:

1. `magic` — 4 bytes ASCII: `DMOR`
2. `version` — 5 bytes ASCII: `0.001`
3. `manifest_encoding` — 1 byte
4. `flags` — 1 byte
5. `manifest_length` — 8 bytes unsigned big-endian
6. `payload_length` — 8 bytes unsigned big-endian
7. `reserved` — 8 bytes unsigned big-endian
8. `manifest_bytes`
9. `payload_bytes`

13.2 Fixed Values

- `magic` **MUST** be `DMOR`
- `version` **MUST** be ASCII `0.001`
- `manifest_encoding` **MUST** be `0x01` for UTF-8 JSON in v0.001
- `flags` **MUST** be `0x00` in v0.001
- `reserved` **MUST** be zero in v0.001

13.3 Length Values

- `manifest_length` **MUST** equal the exact byte length of `manifest_bytes`
- `payload_length` **MUST** equal the exact byte length of `payload_bytes`

13.4 Manifest Content

`manifest_bytes` **MUST** be a valid public manifest JSON object as defined in Section 9.

13.5 Payload Content

`payload_bytes` **MUST** be non-empty.

`payload_bytes` are opaque to this specification. Their internal meaning is defined only by reconstruction objects and implementation-specific cryptographic logic.

---

14. TXT Datamorpho Envelope (DTE-0.001)

The TXT carrier profile uses a terminal textual envelope.

14.1 Required Delimiters

A TXT Datamorphed file **MUST** end with a Datamorpho envelope using these marker lines exactly:

```
===DATAMORPHO-BEGIN===
...
===DATAMORPHO-END===
```

14.2 Placement

The envelope **MUST** appear after the visible base text.

Writers **SHOULD** insert at least three line breaks before `===DATAMORPHO-BEGIN===`.

14.3 Required Envelope Header Fields

Inside the envelope, the following header lines **MUST** appear before the manifest body:

```
Datamorpho-Version: 0.001
Carrier-Profile: txt-envelope
Manifest-Encoding: json-utf8
Payload-Encoding: base64url
Manifest-Length: <decimal>
Payload-Length: <decimal>
```

14.4 Envelope Body Layout

After the header lines, writers **MUST** emit:

1. one blank line,
2. the public manifest JSON text,
3. one blank line,
4. the base64url-encoded payload text,
5. the end delimiter.

14.5 Manifest Body

The manifest body **MUST** be a public manifest JSON object as defined in Section 9.

14.6 Payload Body

The textual payload body **MUST** be base64url without padding.

`Payload-Length` **MUST** specify the byte length of the **decoded** payload byte region, not the encoded character count.

14.7 Canonical Payload Region Name

For TXT files, the single payload region name **MUST** be:

main

---

15. JPEG Carrier Profile (`jpeg-trailer`)

15.1 Recognition

A JPEG Datamorphed file **MUST** be a valid JPEG file followed by a DMB-0.001 block.

15.2 Placement

The DMB-0.001 block **MUST** begin immediately after the final JPEG `EOI` marker bytes `FF D9`.

15.3 Base File Preservation

The bytes from the start of the file through the final `EOI` marker **MUST** remain a valid JPEG stream.

15.4 Public Manifest

The DMB public manifest **MUST** declare:

- `"carrier": "jpeg"`
- `"profile": "jpeg-trailer"`

15.5 Payload Region Name

For JPEG files, the single payload region name **MUST** be:

main

15.6 Detection Rule

A parser identifying the `jpeg-trailer` profile **MUST**:

1. parse the JPEG through `EOI`,
2. check whether the next 4 bytes are `DMOR`,
3. if yes, parse the following bytes as DMB-0.001.

If the bytes after `EOI` do not begin with `DMOR`, the file **MUST NOT** be treated as Datamorpho under this profile.

---

16. PDF Carrier Profile (`pdf-incremental`)

16.1 General Model

A PDF Datamorphed file **MUST** remain a valid PDF and **MUST** store Datamorpho data in an appended incremental update.

16.2 Catalog Link

The updated PDF catalog **MUST** contain a `/Datamorpho` entry referencing a Datamorpho dictionary object.

Example:

```
<<
 /Type /Catalog
 ...
 /Datamorpho 17 0 R
>>
```

16.3 Datamorpho Dictionary Object

The referenced Datamorpho dictionary object **MUST** contain:

```
<<
 /Type /Datamorpho
 /DMVersion (0.001)
 /DMProfile /pdf-incremental
 /DMManifest 18 0 R
 /DMPayloads <<
   /main 19 0 R
 >>
>>
```

Required Entries

- `/Type /Datamorpho`
- `/DMVersion (0.001)`
- `/DMProfile /pdf-incremental`
- `/DMManifest <objref>`
- `/DMPayloads << ... >>`

16.4 Manifest Object

`/DMManifest` **MUST** reference a stream object whose stream bytes are UTF-8 JSON representing the public manifest.

Recommended form:

```
<<
 /Type /DatamorphoManifest
 /DMEncoding /json-utf8
 /Length N
>>
stream
{ ... public manifest JSON ... }
endstream
```

16.5 Payload Stream Objects

Each entry in `/DMPayloads` **MUST** map a payload-region name to a stream object reference.

Example:

```
/DMPayloads <<
 /main 19 0 R
 /state2 20 0 R
>>
```

Each referenced object **MUST** be a stream object containing the raw canonical payload bytes for that region.

Recommended form:

```
<<
 /Type /DatamorphoPayload
 /DMRegion /main
 /Length N
>>
stream
... opaque payload bytes ...
endstream
```

16.6 Region Names

Each `/DMPayloads` key name **MUST** conform to the identifier rules in Section 6 when interpreted without the leading slash.

16.7 Manifest Requirements

For PDF files, the public manifest **MUST** declare:

- `"carrier": "pdf"`
- `"profile": "pdf-incremental"`

16.8 Alternate Complete PDF States

A hidden state MAY itself be a complete alternate PDF. In that case, the concealed payload bytes MAY represent that PDF state in encrypted, fragmented, sparse, or sparse-with-chaff form.

This specification does not define viewer replacement behavior beyond file structure.

---

17. Conformance by Role

17.1 Creator Conformance

A creator implementation conforms to v0.001 if it:

1. generates a valid public manifest per Section 9,
2. generates at least one non-empty payload region,
3. writes the Datamorpho data according to exactly one supported carrier profile,
4. preserves carrier validity,
5. emits payload offsets only relative to canonical payload bytes,
6. includes a `reconstruction_digest` for each state,
7. and does not place reconstruction instructions in the public manifest beyond what this spec allows.

17.2 Reconstruction Object Generator Conformance

A reconstruction-object generator conforms to v0.001 if it:

1. emits valid reconstruction JSON per Section 10,
2. identifies exactly one `state_id`,
3. uses valid offsets and lengths,
4. refers only to defined payload regions,
5. includes a valid `carrier_file_digest`,
6. computes `reconstruction_digest` according to Section 7.8,
7. does not rely on encoded transport length for offset semantics,
8. and includes sufficient key material to reconstruct all fragments of the target state, whether through `state_key_material`, fragment-level `key_material`, or both.

17.3 Parser Conformance

A parser conforms to v0.001 if it:

1. correctly detects the carrier profile,
2. extracts the public manifest,
3. extracts canonical payload regions,
4. tolerates unknown extension fields,
5. rejects malformed required fields,
6. and, when a reconstruction object is provided, validates its `state_id`, region references, and required digests.

---

18. Validation Rules

A Datamorphed file **MUST** be rejected as non-conformant if any of the following occur:

- the public manifest is missing,
- the public manifest is not valid UTF-8 JSON,
- required manifest fields are missing,
- `states` is empty,
- a required `reconstruction_digest` is missing from any state descriptor,
- a required trigger field is missing,
- a required MorphoStorage descriptor field is missing,
- the `morphostorage` array is empty,
- the payload region is empty,
- the carrier is invalid,
- the profile is invalid,
- a binary length field does not match actual bytes.

A reconstruction object **MUST** be rejected as non-conformant if:

- required reconstruction fields are missing,
- `layout_strategy` is not one of the defined values,
- `carrier_file_digest` is missing,
- a fragment references an undefined payload region,
- a fragment has a negative offset,
- a fragment has non-positive length,
- `state_id` does not match a state declared in the manifest,
- or a fragment has no applicable key material, meaning neither `state_key_material` nor fragment-level `key_material` is available for that fragment.

Parsers **MAY** reject additional malformed conditions.

---

19. Versioning Rules

19.1 Version String

The version string for this specification is exactly:

0.001

19.2 Forward Compatibility

Parsers implementing `0.001`:

- **MUST** reject a Datamorpho object whose `datamorpho_version` is not `"0.001"` unless the parser explicitly supports that other version.
- **MUST** ignore unknown `x-*` extension fields.
- **SHOULD** ignore unknown non-critical fields that do not conflict with required semantics.

19.3 Extension Policy

Future versions **SHOULD** preserve:

- public/private metadata separation,
- canonical payload offset semantics,
- digest cross-binding semantics,
- and carrier validity requirements.

---

20. Security Requirements

20.1 General

Implementations **MUST NOT** represent sparse layout, chaff, hidden suite maps, or fragment disorder as substitutes for sound cryptography.

Reconstruction objects are secret-bearing artifacts and **MAY** contain the key material needed to reconstruct the target hidden state. They therefore require protection equivalent to the protection expected for secret reconstruction instructions.

20.2 Required Security Framing

This specification supports:

- cryptographic agility,
- fragment-level compartmentalization,
- non-monotonic physical placement,
- increased attacker workload,
- state-specific release models,
- and split publication of reconstruction objects.

This specification does **not** guarantee:

- absolute secrecy,
- absolute post-quantum permanence,
- or safety in the presence of broken implementations or compromised endpoints.

20.3 Public Manifest Restrictions

The public manifest **MUST NOT** contain:

- fragment offsets,
- fragment lengths,
- fragment order,
- cryptographic suite map details,
- keys,
- or reconstruction instructions.

20.4 Reconstruction Object Publishing

Reconstruction objects **MAY** be independently published and split across channels. This specification does not mandate secrecy transport, signing, or access control.

---

21. Responsible Use Statement

Datamorpho is intended for lawful, ethical, and constructive use, including publishing, games, interactive assets, archives, collectibles, art, research, and controlled disclosure.

This specification does not endorse criminal concealment, fraud, extortion, abuse, unauthorized access, or illegal conduct.

This section is declarative and does not alter technical conformance.

22. Minimal Examples

22.1 Minimal Public Manifest


```
{
 "datamorpho_version": "0.001",
 "manifest_type": "public",
 "carrier": "jpeg",
 "profile": "jpeg-trailer",
 "file_id": "demo-001",
 "states": [
   {
     "state_id": "state-1",
     "triggers": [
       {
         "trigger_id": "t1",
         "type": "time",
         "value": "2026-08-01T00:00:00Z",
         "description": "First release window"
       }
     ],
     "morphostorage": [
       {
         "type": "uri",
         "value": "ipfs://bafyexample"
       },
       {
         "type": "evm",
         "chain_id": 1,
         "contract_address": "0x1234567890abcdef1234567890abcdef12345678",
         "function": "getReconstructionObject",
         "value": "state-1"
       }
     ],
     "reconstruction_digest": {
       "algorithm": "sha256",
       "value": "mL9s4Yxv2L8x9u5dP1d3m8g6n0Q2r4w7t6z1b3e5c7A"
     },
     "label": "First hidden state"
   }
 ]
}
```


22.2 Minimal Reconstruction Object


```
{
  "datamorpho_version": "0.001",
  "object_type": "reconstruction",
  "reconstruction_id": "r1",
  "state_id": "state-1",
  "layout_strategy": "sparse-with-chaff",
```

```
  "carrier_file_digest": {
    "algorithm": "sha256",
    "value": "Qh4YfT8hYQbTz5lK0cZ7mM8uDk2r2u2tX6g3n4k9m2U"
  },
  "state_key_material": {
    "format": "raw",
    "encoding": "base64url",
    "value": "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
  },
  "fragments": [
    {
      "fragment_id": "f1",
      "payload_region": "main",
      "offset": 59,
      "length": 442,
      "order": 1,
      "crypto_suite": "AES-256-GCM"
    },
    {
      "fragment_id": "f2",
      "payload_region": "main",
      "offset": 20,
      "length": 20,
      "order": 2,
      "crypto_suite": "ChaCha20-Poly1305",
      "key_material": {
        "format": "raw",
        "encoding": "base64url",
        "value": "BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB"
      }
    },
    {
      "fragment_id": "f3",
      "payload_region": "main",
      "offset": 900,
      "length": 9,
      "order": 3,
      "crypto_suite": "AES-256-GCM"
    },
    {
      "fragment_id": "f4",
      "payload_region": "main",
      "offset": 1,
      "length": 9,
      "order": 4,
      "crypto_suite": "ML-KEM-768+AES-256-GCM",
      "key_material": {
        "format": "suite-defined",
        "encoding": "base64url",
        "value": "CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC"
      }
    }
  ],
  "reassembly": "concat-by-order"
}
```

22.3 Minimal TXT Envelope

Visible text content.


```
===DATAMORPHO-BEGIN===
Datamorpho-Version: 0.001
Carrier-Profile: txt-envelope
Manifest-Encoding: json-utf8
Payload-Encoding: base64url
Manifest-Length: 486
Payload-Length: 64

{"datamorpho_version":"0.001","manifest_type":"public","carrier":"txt","profile":"txt-
envelope","states":[{"state_id":"state-1","triggers":[{"trigger_id":"t1","type":"custo
m","value":"minted"}],"morphostorage":[{"type":"uri","value":"https://example.org/s1.j
son"}],"reconstruction_digest":{"algorithm":"sha256","value":"mL9s4Yxv2L8x9u5dP1d3m8g6
n0Q2r4w7t6z1b3e5c7A"}}]}

VGhpcyBpcyBhbiBleGFtcGxlIHBheWxvYWQgdGhhdCB3b3VsZCBub3JtYWxseSBiZSBiYXNlNjR1cmwu
===DATAMORPHO-END===
```


---

23. Recommended File Creation Order

The following creation order is RECOMMENDED:

1. select base file,
2. prepare hidden state source,
3. transform hidden state into concealed payload bytes,
4. build reconstruction object **without** file-binding digest fields,
5. compute `reconstruction_digest` using Section 7.8,
6. build public manifest including that digest,
7. write carrier profile,
8. hash the final Datamorphed carrier file,
9. add `carrier_file_digest` to the reconstruction object,
10. publish or store reconstruction objects as needed,
11. validate resulting file and reconstruction objects.

This section is RECOMMENDED, not mandatory.

---

24. Summary of Supported Carrier Profiles in v0.001

Carrier	Profile	Public Manifest Location	Payload Location

JPEG	`jpeg-trailer`	DMB manifest section after `EOI`	DMB payload section after `EOI`
TXT	`txt-envelope`	Inside terminal Datamorpho envelope	Base64url payload inside terminal envelope
PDF	`pdf-incremental`	`/DMManifest` stream object in incremental update	`/DMPayloads` stream object(s) in incremental update

---

25. Final Normative Statement

A file conforms to **Datamorpho Specification v0.001** only if:

- it conforms to one defined carrier profile,
- it contains a valid public manifest,
- it contains at least one non-empty payload region,
- it preserves the validity of the base carrier file,
- and any reconstruction object produced for it follows the canonical offset, layout, and digest semantics defined herein.